AI ผู้ช่วยพนักงานรับเรื่องเคลมประกันรถยนต์

> ***เอกสารออกแบบ POC*** *— AI ที่ช่วย* ***พนักงานรับเคลม*** *(claim officer) ประเมินเคสเบื้องต้นจากข้อมูลที่กรอก/บันทึกไว้ แล้วส่งต่อให้พนักงานตัดสินใจสุดท้าย ตอบ assignment "Motor Insurance Claim Triage Assistant"*
>
> ***ผู้เขียน:*** *Sayomphoo |* ***วันที่:*** *2026-10-28*

สารบัญ

1. เรื่องเล่าและปัญหา
2. AI จะช่วยอะไร / ไม่ช่วยอะไร
3. ใครได้ใช้ และใช้ยังไง
4. ภาพรวมวิธีการทำงาน
5. สถาปัตยกรรมเบื้องหลัง
6. เลือก AI ตัวไหน และทำไม
7. ข้อมูลที่ต้องมี
8. วิธีสั่ง AI ให้ทำงานอย่างถูกต้อง
9. รูปแบบโฟลเดอร์โปรเจกต์
10. วัดผลยังไงว่า AI ทำงานดี
11. ความเสี่ยงและวิธีป้องกัน
12. แผนทำจริง (MVP → เต็มรูปแบบ)
13. ลำดับการพรีเซนต์

---

1. เรื่องเล่าและปัญหา

1.1 เรื่องเล่าจากประสบการณ์จริง (มุมมองลูกค้า → ต้นเหตุที่ officer ต้อง triage ช้า)

เรื่องที่ 1 — สายเกียร์รถขาด

> *วันนั้นผมกำลังถอยรถออกจากบ้านด้วยเกียร์ R เมื่อถึงถนนและเปลี่ยนเป็นเกียร์ D ทันใดนั้นสายเกียร์ขาด เกิดเสียงดัง แต่ยังเคลื่อนรถไปจอดข้างทางได้ หลังเข้าเกียร์ P แล้ว รถเข้าเกียร์อื่นไม่ได้อีกเลย*
>
> *ผมโทรเข้า call center และส่งเอกสารตามที่พนักงานขอ แต่กว่าพนักงานจะเปิดเล่ม policy อ่าน + ตรวจประวัติ + ประเมินว่าควรส่งเคสไปไหน* ***ใช้เวลาเป็นชั่วโมง*** *ผมเองก็ไม่รู้ว่าเคสจะได้รับหรือไม่ จนกว่าจะมีคนโทรกลับ*

เรื่องที่ 2 — เคลมสเกิตหน้า/หลังรถ

> *สเกิตรถของผมบิ่นและแตก ส่งเอกสารครบตามที่ประกันขอ แต่คำตอบว่า "จะเข้าได้ไหม" ต้องรอ* ***พนักงานรีวิวทีละเคส*** *ซึ่งเคสในวันนั้นมีจำนวนมาก*

1.2 สรุปปัญหาที่เจอ (Pain Points)

#	ปัญหา	ใครเจ็บ	ส่งผลยังไง
P1	พนักงานต้องอ่าน policy + ตรวจเอกสาร + สรุปเองทุกเคส	พนักงาน	AHT (average handle time) นาน · รับเคสได้น้อย/วัน
P2	แต่ละพนักงานตีความ policy ไม่เท่ากัน	ลูกค้า + บริษัท	ลูกค้าได้คำตอบไม่สม่ำเสมอ · ข้อร้องเรียน
P3	จับสัญญาณฉ้อโกง / exclusion ได้ช้า	บริษัท	เสียเงินโดยไม่ควร
P4	ลูกค้าส่งเอกสารไม่ครบ กว่าจะรู้ต้องรอพนักงานรีวิวก่อน	ลูกค้า	เสียเวลาส่งไป-กลับหลายรอบ
P5	ไม่มี audit trail ชัด ว่าแต่ละ decision อิง clause ไหน	ฝ่าย compliance	ตอบ regulator ยาก
P6	เคสง่ายกับเคสยากมารวมคิวเดียวกัน	พนักงาน	เคสยากถูก delay · เคสง่ายไม่ auto

1.3 แก้ปัญหาแต่ละข้อยังไง

ตารางนี้สำคัญ — ผู้ตรวจอยากเห็นว่าทุกปัญหามีคำตอบ และวัดได้ว่าแก้ได้จริง

ปัญหา	AI จะช่วยยังไง	จะวัดยังไงว่าแก้ได้
P1 (manual review นาน)	AI สรุปเคส + ประเมินเบื้องต้น + บอกว่าควรส่งไปทีมไหน พร้อม cite clause	AHT ↓ 30-40%
P2 (ประเมินไม่เท่ากัน)	AI ใช้ prompt + guardrails เดียวกันทุกเคส	routing consistency ≥ 0.75 (รัน eval ซ้ำได้ผลใกล้เคียง)
P3 (จับฉ้อโกงช้า)	AI ตรวจ pattern (เคลมถี่ · เอกสารอ่อน · story ขัดแย้ง) ตั้งแต่ครั้งแรก	fraud flag recall ↑ 15%
P4 (ขาดเอกสาร)	AI บอก missing documents ตั้งแต่ตอนรับเคสเข้า	documents incomplete loop ↓ 50%
P5 (ไม่มี audit)	ทุก decision เก็บ clause + model version + timestamp	100% ของเคสมี audit JSON
P6 (เคสง่าย/ยากปน)	AI route Standard / Manual / Fraud / Rejection อัตโนมัติ	% เคสที่ไปเข้า standard processing อย่างถูกต้องสูง

1.4 ประโยชน์ที่คาดว่าจะได้

- **พนักงาน:** ใช้เวลาต่อเคสน้อยลง · รับเคสได้มากขึ้น/วัน · ไม่ต้องเปิดเล่ม policy ทุกครั้ง
- **ลูกค้า:** ได้คำตอบเร็วขึ้น · ได้ feedback เอกสารขาดครั้งเดียว ไม่ส่งไปมา
- **ความเสี่ยง:** จับฉ้อโกงหรือข้อยกเว้นได้ตั้งแต่ครั้งแรก → ลดเงินจ่ายที่ไม่ควรจ่าย
- **การตรวจสอบ:** ทุกการตัดสินใจมี citation + audit → ตอบ regulator ได้

---

2. AI จะช่วยอะไร / ไม่ช่วยอะไร

2.1 ขอบเขตที่ AI รับผิดชอบ (ตรงกับ PDF §4 In Scope)

AI ทำ

- รับข้อมูลเคสที่ officer กรอก (ClaimInput ตาม PDF §5.2)
- สรุปเคสเป็น claim_summary สั้นๆ
- ประเมิน coverage เบื้องต้น (Likely / Possibly / Not / Cannot determine)
- บอก missing information ตาม Section 3 ของ policy
- Flag risk patterns (ตาม Clause 4.3)
- แนะนำ routing (Standard / Manual / Fraud / Rejection — Clause 4.1-4.4)
- Cite clause ที่ใช้ตัดสินใจ ([Clause X.Y]) ทุกข้อสรุป
- ให้ confidence level (High/Medium/Low)

AI ไม่ทำ (ตรงกับ PDF §4 Out of Scope)

- ตัดสินใจอนุมัติ/ปฏิเสธเคลมเอง — ต้องให้คนตัดสิน
- คำนวณจำนวนเงินสินไหม
- สื่อสารกับลูกค้าโดยอัตโนมัติ (คนต้องตรวจก่อนส่งออก)
- เชื่อมกับระบบ core จริงของบริษัท
- ฝึกโมเดลจับฉ้อโกงระดับโปรดักชัน
- ทำ security hardening ระดับองค์กร

2.2 เส้นแบ่งการตัดสินใจที่สำคัญ

> ***AI เป็น "ผู้ช่วยพนักงาน" ไม่ใช่ "ผู้ตัดสินแทน"***
>
> - *AI แนะนำ coverage + routing + cite clause → **พนักงานเป็นผู้ตัดสินใจสุดท้าย***

> - *ถ้าข้อมูลไม่พอ → AI ตอบ `Cannot determine` + confidence ต่ำ แทนที่จะเดา*
> - *Low confidence บังคับ route ไป **Manual review** เสมอ (ห้ามไป Standard processing)*

2.3 วัดความสำเร็จอย่างไร (บน 5 PDF test cases)

เรื่องที่วัด	เป้าหมาย	ผลจริง
Routing ถูกต้อง (เลือก 1 ใน 4)	≥ 85%	**100%** (5/5) ⭐
Coverage ถูกต้อง (เลือก 1 ใน 4)	≥ 80%	80% (4/5)
Citation validity (clause ที่ cite มีจริงใน retrieved)	100%	**100%** ⭐
Missing docs identification F1	≥ 0.70	0.87
Risk flags identification F1	≥ 0.70	0.75
LLM-Judge mean score (reasoning quality)	≥ 3.5/5	**4.00/5**

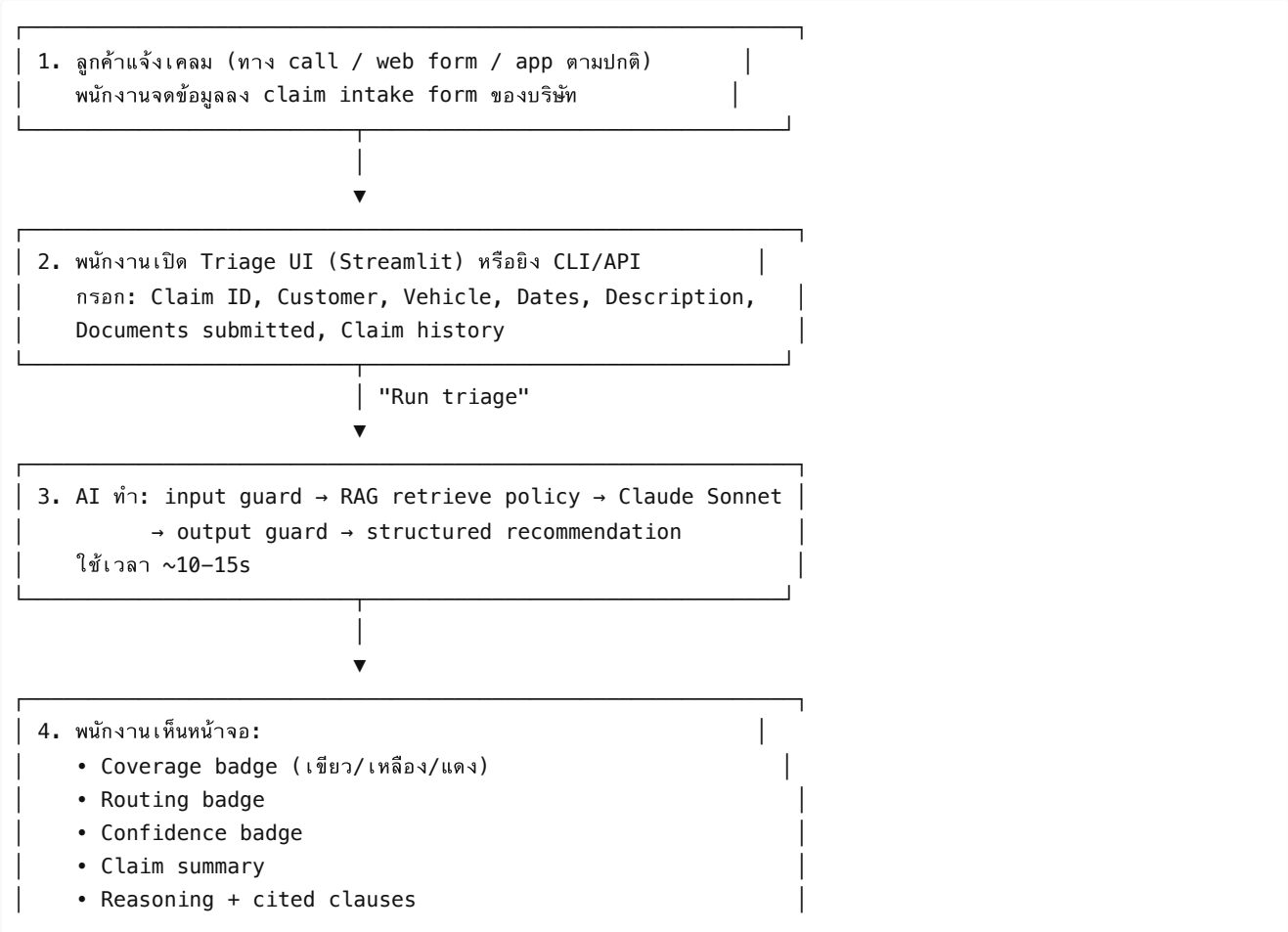
---

3. ใครได้ใช้ และใช้ยังไง

3.1 คนที่เกี่ยวข้อง

- **พนักงานรับเคลม (ผู้ใช้หลัก)** — เปิด UI กรอกข้อมูลเคสที่ได้จากลูกค้า → ดู triage recommendation → ตัดสินใจต่อ
- **หัวหน้าทีม** — ดูแดชบอร์ด audit log · ตรวจเคสยาก · spot-check sample
- **ฝ่ายกฎหมาย / Compliance** — ดูบันทึก audit · ตรวจว่าทุก decision cite clause จริง · ตอบ regulator
- **ลูกค้า (ไม่ตอบโต้ AI โดยตรง)** — ลูกค้าให้ข้อมูลตามปกติผ่าน call center / web form / app พนักงานเป็นคนป้อนข้อมูลเข้าระบบ

3.2 User Journey (ฝั่งพนักงาน)



```
│    • Missing information list                                  │
│    • Risk flags list                                           │
│    • Retrieved policy clauses (expand ดูได้)                   │
│    • Guardrail triggers (expand ดูได้)                         │
│    • Audit record JSON (expand ดูได้)                          │
└──────────────────────────┬─────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 5. พนักงานตัดสินใจ:                                          │
│    • ยืนยัน routing ตามที่ AI แนะนำ → ส่งต่อตามทีม          │
│    • หรือ override routing + บันทึกเหตุผล                    │
│    • หรือขอข้อมูลเพิ่มจากลูกค้า (ตาม missing_info list)      │
└──────────────────────────────────────────────────────────────┘
```

3.3 ตัวอย่างหน้าจอที่พนักงานเห็น (จาก Streamlit UI)

```
┌──────────────────────────────────────────────────────────────┐
│ Claim ID: CLM-2026-00123                                     │
│ Customer: C-14201 · Vehicle: Toyota Camry 2.5G 2022          │
│ ──────────────────────────────────────────────────────────── │
│ [Coverage: Possibly covered] [Routing: Manual review]        │
│ [Confidence: Medium]                                         │
│                                                              │
│ Claim summary:                                               │
│   Customer's parked vehicle was struck by a reversing        │
│   third-party driver at Central World mall. Note left but    │
│   driver departed. Submitted 2 days after incident.          │
│                                                              │
│ Reasoning:                                                   │
│   Event matches third-party property damage [Clause 1.1].    │
│   Missing driving license, incident report [Clause 3.1].     │
│   Third-party contact and evidence missing [Clause 3.3].     │
│   Cannot proceed to standard processing [Clause 4.1].        │
│   Route to manual review [Clause 4.2].                       │
│                                                              │
│ Missing information:                                         │
│   • Driving license copy (Clause 3.1)                        │
│   • Incident report (Clause 3.1)                             │
│   • Third-party contact (Clause 3.3)                         │
│   • Third-party evidence (Clause 3.3)                        │
│                                                              │
│ Risk flags:                                                  │
│   • Third-party departed scene — identity unverified         │
│                                                              │
│ Cited: Clause 1.1, 3.1, 3.3, 4.1, 4.2                        │
│                                                              │
│ [ ▸ Retrieved policy clauses (5)                          ]  │
│ [ ▸ Guardrail triggers (input: 0, output: 0)              ]  │
│ [ ▸ Audit record — sess_abc123                            ]  │
└──────────────────────────────────────────────────────────────┘
```

---

4. ภาพรวมวิธีการทำงาน

ลองนึกภาพว่า AI นี้ทำงานเหมือน **"ผู้ช่วยพนักงาน"** ที่:

1. **รับข้อมูลเคส** — จาก Streamlit form (หรือ CLI, API) ที่พนักงานกรอก
2. **กัน PII ก่อน LLM เห็น** — mask เบอร์โทร, เลขบัตรประชาชน, ทะเบียน, email
3. **เปิดเล่ม policy** — ค้น clause ที่เกี่ยวข้องผ่าน RAG (4 sub-queries คนละ section)
4. **ประเมินตามหลักฐาน** — Claude Sonnet อ่าน policy chunks + ข้อมูลเคส + ตอบใน structured output
5. **ตรวจก่อนส่งต่อ** — Output guardrail ตรวจว่า clause ที่ cite มีจริง + safety invariants
6. **ส่งผลให้พนักงาน** — ขึ้นหน้าจอพร้อม cite + confidence + missing docs + risk flags
7. **บันทึก audit** — เก็บ model version, clauses, latency, guard triggers ลง JSON ต่อ session

ทำไมใช้ RAG + grounded prompt ไม่ใช้ LLM ตรงๆ?

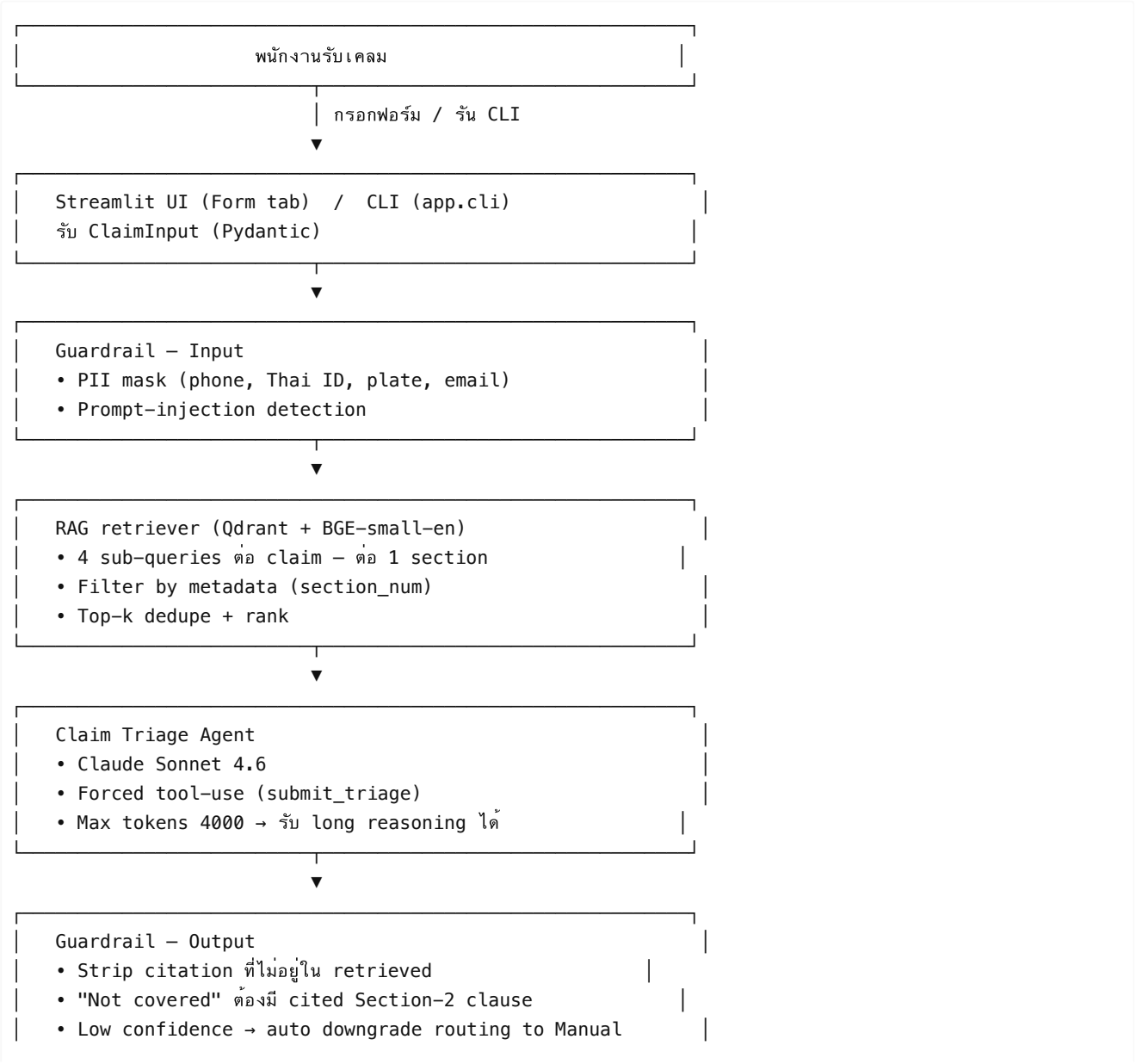
เพราะ:

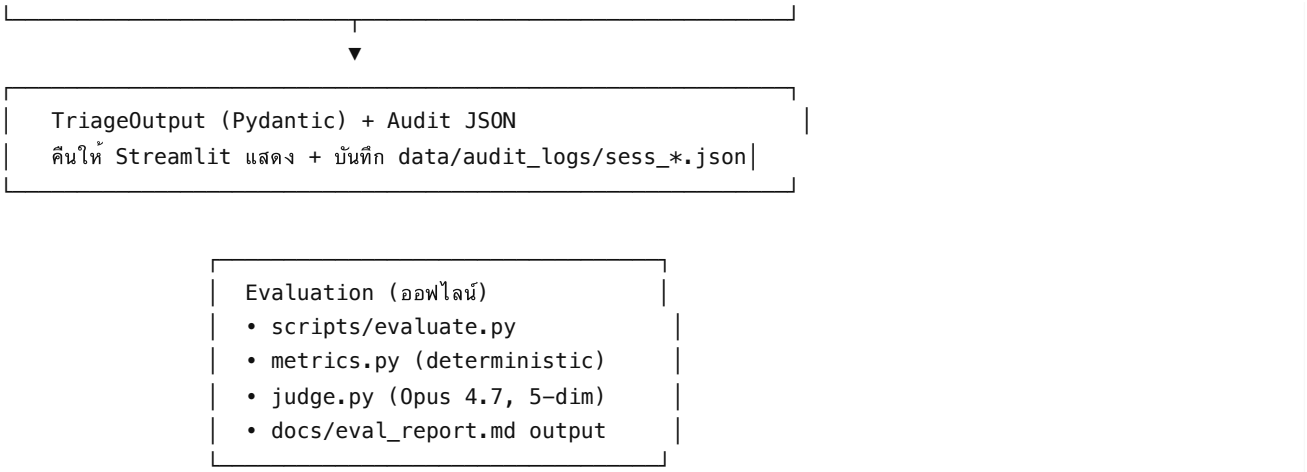
- **Policy เปลี่ยนได้** — ถ้า hardcode ลง prompt ต้องแก้ code ทุกครั้ง RAG = อัปเดต markdown ไฟล์ + re-ingest
- **Audit ต้องชัด** — cite ที่มาจาก retrieved context เท่านั้น ไม่งั้น reviewer ตามไม่ได้
- **Scale clause ได้** — ตอนนี้ 13 clauses · ถ้า policy โตเป็นร้อย clauses semantic search คุ้มกว่า dumping prompt
- **ป้องกัน hallucination** — output guardrail ตรวจ cite กับ retrieved context ตรงๆ

---

5. สถาปัตยกรรมเบื้องหลัง

5.1 ภาพรวม (แบบย่อย่อ)





5.2 ส่วนที่ Rubric อยากเห็น

ส่วน Standard (ที่ต้องมี):

หัวข้อ	เราทำยังไง	ทำไมสำคัญ
แปลงข้อมูลเป็น Vector เพื่อ RAG	Policy markdown → อ่าน section-aware chunks (1 clause = 1 chunk) → embed ด้วย BGE-small-en-v1.5 → เก็บ Qdrant	ให้ AI หา clause ที่เกี่ยวข้องได้แม่น
ตั้ง parameter ตอนค้น	4 sub-queries ต่อ claim (section-filtered), top-k=5 ต่อ query, dedupe + merge	1 query ธรรมดามีโอกาสพลาด Section 2 (exclusions) → sub-query ต่อ section การันตีครอบคลุม
ถ้าแม่นต่ำแก้ยังไง	ใส่ **metadata** `section_num` ให้ filter ก่อนค้น + 4 sub-queries	ประหยัด API cost + เพิ่มความแม่น
ข้อจำกัดโมเดล	Claude Sonnet 4.6 รับ context ยาว แต่เราจำกัดโดยส่ง top-k=5 chunks เท่านั้น	สมดุล quality + cost
ใช้ Framework	Pydantic schemas + Anthropic SDK tool-use (ไม่ใช้ LangGraph เพราะ workflow เป็น linear 1 ขั้น)	โค้ดสั้น · debug ง่าย · ไม่พึ่ง abstraction เกิน
ห่อเป็น Agent	TriageAgent class ที่ embed guardrails + retriever + Claude call + audit	ทำงาน end-to-end ด้วย 1 method `agent.run(claim)`

ส่วน Plus (ถ้ามีจะได้คะแนนเพิ่ม) — อยู่ใน `extensions` branch:

หัวข้อ	สถานะใน `develop`	สถานะใน `extensions`
Multi-Agent	❌ (Phase 2)	✅ SupervisorAgent → IntakeSpecialist + TriageSpecialist
ข้อมูลไม่ใช่ภาษาอังกฤษ (Thai)	❌ (Phase 2)	✅ Multilingual e5-large + 14 bilingual test cases
Voice (STT + TTS)	❌ (Phase 2)	✅ Whisper + Edge-TTS

---

6. เลือก AI ตัวไหน และทำไม

6.1 ไม่ใช้ LLM ทำทุกอย่าง — ผสมกฎกับ LLM

งาน	ใช้อะไร	เหตุผล

ประเมิน coverage ตาม policy	Claude Sonnet 4.6 + RAG	ต้องใช้เหตุผล + cite policy
ตัดสินใจ routing	Claude Sonnet 4.6 อ้างอิง Clause 4.1-4.4	กฎใน policy ชัดอยู่แล้ว แต่ยังมี ambiguity (เช่น "unclear coverage") → ใช้ LLM สนับสนุน
ตรวจสัญญาณเสี่ยง (Clause 4.3)	Claude Sonnet 4.6 + heuristic prompt	repeated claims / weak evidence / inconsistent story — LLM จับ pattern ได้
ตรวจเอกสารครบ	Claude Sonnet 4.6 เทียบกับ Clause 3.1-3.3	รายการเอกสาร = check list ง่าย แต่ map กับประเภทเหตุต้องใช้ LLM
Judge (ประเมิน reasoning quality)	**Claude Opus 4.7**	คนละ model family กับ Sonnet → ลด self-grading bias
Guardrail ตรวจ cite	**กฎ** (ไม่ใช้ LLM)	ตรวจ set membership ใน retrieved IDs ไม่ต้องใช้ AI

กฎทอง: ใช้ AI เฉพาะเรื่องที่ตัดสินใจตายตัวไม่ได้ ส่วนที่กฎชัด (เช่น ตรวจ citation validity) → ใช้กฎ ประหยัดและไม่มั่ว

6.2 Tech Stack (เลือกใช้แต่ละตัวเพราะอะไร)

ชั้น	เลือกใช้	ทำไมเลือก
สมอง AI (คิด)	Claude Sonnet 4.6	เก่งเหตุผล · tool-use adherence สูง · Thai + EN · context ยาว
Judge	Claude Opus 4.7	คนละ family กับ Sonnet · ลด bias
แปลงข้อความ → ตัวเลข	BAAI/bge-small-en-v1.5 (384-dim, 33MB)	ขนาดเล็ก · เพียงพอสำหรับ 13 clauses · เร็ว
ที่เก็บข้อมูลตัวเลข	Qdrant	เร็ว · filter metadata ได้ · open source · docker-friendly
Structured output	Pydantic + Anthropic tool-use	ไม่มี free-form prose · enum validation + shape guarantee
Backend/UI	FastAPI pattern + Streamlit	prototyping เร็ว · reviewer-friendly
Observability	Rich (console) + JSON audit log	lightweight · ไม่พึ่ง SaaS

> ***หมายเหตุ:*** *Design doc เดิมเคยคิดใช้ Whisper + ElevenLabs + Langfuse + NeMo Guardrails ปัจจุบัน POC* `develop` *scope ตรง PDF text-based → ตัดออกเพื่อโฟกัส component ที่ PDF ประเมิน Voice stack อยู่ใน* `extensions` *branch*

---

7. ข้อมูลที่ต้องมี

7.1 แหล่งข้อมูล

แหล่ง	รูปแบบ	ที่อยู่ในโปรเจค
Motor Insurance Policy Rules (PDF §5.1)	Markdown + Clause IDs	`data/policies/motor_policy.md`
Claim Input Fields (PDF §5.2)	JSON	`data/test_cases/case*.json`
Ground truth สำหรับ eval	JSON	`data/eval/ground_truth.json`
Audit log output	JSON ต่อ session	`data/audit_logs/sess_*.json` (gitignored)

7.2 Policy Rules structure (ตาม PDF §5.1 เป๊ะ)

```
Section 1 — Covered Events
  Clause 1.1  — Covered Perils
Section 2 — Exclusions
  Clause 2.1  — Alcohol or Drug Impairment
  Clause 2.2  — Illegal Racing
  Clause 2.3  — Intentional Damage
  Clause 2.4  — Geographic Coverage
  Clause 2.5  — Late Submission (>30 days)
Section 3 — Required Documents
  Clause 3.1  — Standard (claim form, license, registration, photos, incident report)
  Clause 3.2  — Theft (+ police report)
  Clause 3.3  — Third-party (+ contact info + evidence)
Section 4 — Routing Rules
  Clause 4.1  — Standard processing
  Clause 4.2  — Manual review
  Clause 4.3  — Fraud review
  Clause 4.4  — Rejection review
```

Clause IDs เพิ่มขึ้นมาเพื่อให้ AI cite ได้ตรง (PDF content เดิมไม่มี X.Y numbering)

7.3 คุณภาพข้อมูลที่ต้องระวัง

- **ปิดบัง PII** ก่อนส่งเข้า LLM — เบอร์โทร, เลขบัตร, ทะเบียน, email (ทำโดย input guardrail)
- **เวอร์ชัน policy** — ไฟล์ markdown มี `Policy version: v1.0, effective 2026-01-01` ชัด
- **Context relevance** — top-k=5 สำหรับ 13 clauses อาจเกิน 1/3 ของ corpus หากขยาย policy ต้องปรับ

7.4 Flow ตอนค้น (RAG)

```
Policy markdown ──► parse by `### Clause X.Y` headings ──► 1 clause = 1 chunk
                                                                  │
                                                                  ▼
                                        attach metadata {section_num, clause_id, section, title}
                                                                  │
                                                                  ▼
                                        embed with BGE-small-en-v1.5 (384-dim)
                                                                  │
                                                                  ▼
                                                          Qdrant upsert

ClaimInput ──► 4 section-filtered sub-queries:
    "Covered perils for: {description}"              filter section_num="1"
    "Exclusions that might apply to: {description}"  filter section_num="2"
    "Required docs. Submitted: {docs}"               filter section_num="3"
    "Routing rules for: {description}"               filter section_num="4"
                    │
                    ▼
            Each → top-k=5 ──► dedupe + sort by score ──► ส่งให้ Sonnet
```

7.5 ถ้าแม่นต่ำแก้ยังไง (ตาม rubric)

1. **หาไม่เจอ** → เพิ่ม top-k หรือเพิ่ม sub-queries
2. **context มีขยะ** → tighten section filter · ใช้ reranker
3. **หยิบ policy version ผิด** → metadata filter `policy_version`
4. **AI มั่ว** → บังคับ cite + output guardrail ตรวจ clause ที่ cite มีจริง (ทำแล้ว · ผลลัพธ์: 100% citation validity)

---

8. วิธีสั่ง AI ให้ทำงานอย่างถูกต้อง

8.1 Prompt ของ Claim Triage Agent (ตาม PDF §6 output format)

ไฟล์: `app/prompts/claim_triage.md`

โครงสร้าง XML-style แบ่งเป็น 5 ส่วน:

```
<role>
คุณคือผู้ช่วยพนักงานประเมินเคสเบื้องต้น
**คุณไม่ใช่ผู้ตัดสิน** — พนักงานเป็นคนตัดสินสุดท้าย
</role>

<policy_context>
{retrieved_policy_chunks_with_citation}
</policy_context>

<claim_case>
Claim ID, Customer, Vehicle, Dates, Days to submit,
Documents submitted, Claim history, Description
</claim_case>

<rules>
1. ทุกข้อสรุปเรื่อง coverage/exclusion ต้อง cite [Clause X.Y]
2. ห้ามคาดเดาข้อเท็จจริงที่ไม่มีใน claim
3. Coverage enum: Likely/Possibly/Not/Cannot determine ตามเงื่อนไข
4. Routing: Standard (4.1) / Manual (4.2) / Fraud (4.3) / Rejection (4.4)
5. Low confidence → Manual review (ห้าม Standard)
6. Reasoning ≤ 6 ประโยค
7. ห้ามคำนวณเงิน
8. ไม่ให้คำปรึกษาลูกค้าโดยตรง — audience คือ officer
</rules>

<output_format>
Call submit_triage tool exactly once with structured decision.
</output_format>
```

8.2 Workflow แบบ linear (ไม่พึ่ง LangGraph เพื่อเลี่ยง over-abstraction)

```
[ClaimInput]
    │
    ▼
[Input guardrail] ──► mask PII + flag injection
    │
    ▼
[RAG retrieve] ──► 4 sub-queries ──► top-k=5 ──► merged chunks
    │
    ▼
[Build prompt] ──► inject chunks + case fields
    │
    ▼
[Claude Sonnet 4.6] ──► forced tool_choice=submit_triage
    │
    ▼
[Output guardrail] ──► verify cites + safety invariants
```

```
    │
    ▼
[Audit log write] ──► data/audit_logs/sess_*.json
    │
    ▼
[Return TriageResult]
```

8.3 ถ้าเจอปัญหาต้องทำยังไง (Fallback)

สถานการณ์	ทำยังไง
LLM คืน tool payload ไม่ครบ fields	retry 1 รอบ · ถ้ายังไม่ครบ raise error (observed quirk; max_tokens=4000 ลดโอกาส)
RAG หา clause ไม่เจอ	ส่ง chunks ที่ score สูงที่สุดเท่าที่มี · prompt rule บอก "ถ้าไม่มีให้ตอบ Cannot determine"
Clause cite ไม่มีใน retrieved	output guardrail strip ออก + log trigger
"Not covered" ไม่มี Section-2 clause	downgrade เป็น "Cannot determine" + Low confidence + Manual review
Low confidence + Standard processing	force Manual review

---

9. รูปแบบโฟลเดอร์โปรเจกต์

9.1 โครงสร้างโฟลเดอร์ (develop branch)

```
motor-claim-triage-ai/
├── README.md                          ← วิธีติดตั้ง รัน + branch guide
├── ai-insurance-claim-triage-assistant.md  ← solution design (ไฟล์นี้)
├── ai-insurance-claim-triage-assistant.pdf ← rendered PDF
├── Dockerfile
├── docker-compose.yml                 ← Qdrant service
├── .env.example                       ← config placeholders (no keys)
├── Makefile                           ← install/up/ingest/demo/eval/ui
├── pyproject.toml                     ← uv-managed deps
├── requirements.txt                   ← exported for reviewers
│
├── app/
│   ├── cli.py                         ← python -m app.cli --case ...
│   ├── schemas.py                     ← Pydantic ClaimInput + TriageOutput
│   ├── agents/
│   │   └── triage.py                  ← TriageAgent (end-to-end)
│   ├── rag/
│   │   ├── ingest.py                  ← parse policy + embed + upsert
│   │   └── retriever.py               ← semantic search + section filter
│   ├── prompts/
│   │   └── claim_triage.md            ← main prompt
│   ├── guardrails/
│   │   ├── input_filter.py            ← PII mask + injection detect
│   │   └── output_filter.py           ← cite verify + safety invariants
│   ├── observability/
│   │   └── audit.py                   ← JSON audit log writer
│   ├── eval/
│   │   ├── metrics.py                 ← deterministic metrics
```

```
│   │   └── judge.py                    ← Opus 4.7 rubric scorer
│   └── ui/
│       └── main.py                     ← Streamlit (Form tab)
│
├── data/
│   ├── policies/motor_policy.md        ← PDF §5.1 reformatted
│   ├── test_cases/case1..5_*.json      ← PDF §7 fixtures
│   └── eval/
│       ├── ground_truth.json           ← expected answers per case
│       └── results.json                ← eval harness output
│
├── scripts/
│   ├── ingest_policy.py                ← make ingest wrapper
│   ├── evaluate.py                     ← make eval wrapper
│   └── smoke_test.py                   ← API + Qdrant reachability
│
├── tests/                              ← pytest (unit + integration)
│
└── docs/
    ├── implementation_plan.md          ← 3-day build plan
    ├── eval_report.md                  ← per-case eval results
    ├── limitations.md                  ← trade-offs + Phase 2 roadmap
    ├── prompts.md                      ← prompt design doc
    └── slides/
        ├── presentation.md             ← Marp slide deck
        └── presentation.pdf            ← rendered PDF
```

9.2 ตัวอย่างโค้ดสำคัญ

บังคับให้ output ตรงรูปแบบ (Pydantic + Anthropic tool-use):

```python
class TriageOutput(BaseModel):
    claim_summary: str
    coverage_assessment: Literal["Likely covered", "Possibly covered",
                                 "Not covered", "Cannot determine"]
    missing_information: list[str]
    risk_flags: list[str]
    recommended_routing: Literal["Standard processing", "Manual review",
                                 "Fraud review", "Rejection review"]
    reasoning: str
    confidence_level: Literal["High", "Medium", "Low"]
    cited_clauses: list[str]  # extra field permitted by PDF §6
```

Agent call พร้อม forced tool-use:

```python
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=4000,
    tools=[TRIAGE_TOOL],
    tool_choice={"type": "tool", "name": "submit_triage"},
    messages=[{"role": "user", "content": prompt}],
)
payload = next(b.input for b in response.content if b.type == "tool_use")
output = TriageOutput.model_validate(payload)   # Pydantic guarantees shape
```

9.3 ส่วนที่อยู่ใน `extensions` branch (ไม่อยู่ใน `develop`)

- `app/agents/supervisor.py` + `app/agents/conversation.py` (multi-agent intake)
- `app/voice/stt.py` + `app/voice/tts.py` (Whisper + Edge-TTS)
- `app/observability/call_log.py` (call transcript log)
- `app/ui/main.py` Call tab (voice intake UX)
- `app/prompts/conversation.md`
- `docs/call_agent_playbook.md`
- Thai test cases + multilingual embeddings
- deps: faster-whisper, edge-tts, pydub

---

10. วัดผลยังไงว่า AI ทำงานดี

10.1 ทำไมต้องผสมหลายวิธี

Output บางส่วนเป็นข้อความอิสระ (reasoning, claim_summary) → match keyword ไม่สะท้อนคุณภาพจริง ต้องใช้ 3 วิธีผสม:

สิ่งที่วัด	วิธีวัด	ตรวจอัตโนมัติไหม
Coverage assessment (1 ใน 4)	exact-match กับ accept-any-of set	✅
Routing (1 ใน 4)	exact-match + confusion matrix	✅
Confidence level	exact-match	✅
Missing docs (list)	set-based Precision/Recall/F1 (substring match)	✅
Risk flags (list)	set-based Precision/Recall/F1	✅
Citation validity	ตรวจว่า clause ที่ cite มีจริงใน retrieved	✅
Reasoning quality	**LLM-as-Judge (Opus 4.7) · 5-dim rubric 1-5**	✅ (API cost)

10.2 Rubric ที่ให้ Opus ตัดสิน (จาก `app/eval/judge.py`)

```
ให้คะแนน Reasoning 1–5 ใน 5 มิติ:
1. Factuality — cite clause จริงเท่านั้น
2. Completeness — ครอบคลุม covered peril + exclusions + missing docs + risk flags
3. Clarity — concise (≤ 6 ประโยค) · ไม่พูดเวิ่นเว้อ
4. Policy alignment — routing + coverage ตรงกับ clause ที่ cite
5. Appropriate uncertainty — ใช้ Cannot determine / Low confidence ถูกจังหวะ
```

10.3 Test cases 5 เคส (ตาม PDF §7)

ตัวเลขจากการรัน `make eval` ครั้งล่าสุด:

#	เคส	AI ให้ Coverage	AI ให้ Routing	ตรง GT ไหม?	Judge mean
1	Parked car hit (missing third-party info)	Possibly covered	Manual review	✅ Routing + Coverage	4.00
2	Illegal street racing	Not covered	Rejection review	✅ ทั้งคู่	4.40
3	Theft no police report	Possibly covered	Manual review	✅ ทั้งคู่	4.00

4	Fraud pattern (4 claims/12mo + weak evidence)	Cannot determine	Fraud review	✅ ทั้งคู่	3.40
5	Flood 45 days late submission	Possibly covered	Manual review	⚠️ Routing ✓ Coverage ✗ (คาด Not covered / Cannot determine)	4.20

สรุป aggregate:

- Coverage: **80%** (4/5)
- Routing: **100%** (5/5) ⭐
- Citation validity: **100%** (ไม่มี hallucinated clause) ⭐
- Judge mean: **4.00 / 5**

10.4 Failure mode ที่ยังไม่แก้ (ยอมรับไว้)

- Case 5 (late submission 45 วัน) — AI มักเลือก "Possibly covered" ที่ optimistic ไป เพราะ Clause 2.5 บอกว่า "if valid reason provided, may be considered through manual review" → AI มองว่ายังไม่สรุปได้ว่าไม่คุ้มครอง ควรแก้โดยเพิ่ม deterministic rule ใน prompt: "ถ้า days_to_submit > 30 AND no valid reason in description → coverage = Cannot determine, confidence = Low" documented ไว้ใน `docs/eval_report.md §Qualitative Analysis`

---

11. ความเสี่ยงและวิธีป้องกัน

11.1 ตารางความเสี่ยง

ความเสี่ยง	หนัก?	ป้องกันยังไง (บน develop)
AI มั่ว coverage ไม่มีจริง	🔴 สูง	1) บังคับ cite ทุกข้อสรุป 2) output guardrail ตรวจ cite 3) prompt bias ให้ตอบ "Cannot determine"
PII รั่วเข้า log	🔴 สูง	1) regex mask ก่อนส่ง LLM 2) audit log hash customer
Prompt injection	🟡 กลาง	1) input guard flag injection patterns 2) forced tool-use ล็อก shape output
Routing ผิดทีม	🟡 กลาง	1) prompt rule ชัด (Clause 4.1-4.4) 2) Low confidence → Manual review บังคับ 3) officer override ได้
ค่าใช้จ่ายพุ่ง	🟡 กลาง	1) max_tokens budget ชัด 2) ไม่ยิง Judge นอก eval scope
ตอบช้า	🟡 กลาง	1) latency log ใน audit · เป้า p95 2) Streamlit spinner
ตรวจย้อนไม่ได้	🟡 กลาง	1) ทุก decision cite + audit JSON 2) model version ใน audit

11.2 Guardrails 3 ชั้น (implemented)

```
ชั้น 1: ก่อนให้ AI (Input — app/guardrails/input_filter.py)
  • mask phone / Thai ID / plate / email
  • detect prompt injection patterns

ชั้น 2: ระหว่างทำงาน (schema validation)
  • Pydantic ClaimInput ตรวจ shape
  • Anthropic tool_choice บังคับ submit_triage

ชั้น 3: ก่อนส่งต่อ (Output — app/guardrails/output_filter.py)
```

- strip clause citations ที่ไม่อยู่ใน retrieved
- "Not covered" ต้องมี Section-2 clause cited → ไม่งั้น downgrade
- Low confidence → ห้าม Standard processing → force Manual

11.3 คนต้องอยู่ในลูปตรงไหน (HITL)

จุด	คนทำอะไร
ทุกเคส	officer เห็น recommendation + ตัดสินใจส่งต่อ (ไม่มี auto-approve)
Low confidence	guardrail บังคับ Manual review — officer รีวิวแน่นอน
Not covered	guardrail ตรวจว่ามี exclusion cited จริง — ไม่งั้น downgrade
Fraud review	officer senior ต้องตรวจก่อนปฏิเสธ
ทบทวนรายสัปดาห์	sample 5% + edge cases → feedback เข้า prompt iteration

11.4 Audit record format (ตาม solution-design §11.4 เดิม, implemented)

```
{
  "session_id": "sess_abc123",
  "timestamp": "2026-10-28T14:30:00+07:00",
  "claim_id": "CLM-2026-00123",
  "customer_hash": "sha256:...",         // ไม่เก็บชื่อ raw
  "model_versions": {"triage": "claude-sonnet-4-6"},
  "input_guard_triggers": ["PHONE:0812..."],
  "output_guard_triggers": [],
  "retrieved_chunks": [{"clause_id": "1.1", "score": 0.87}, ...],
  "triage_output": {...},                 // full TriageOutput
  "cited_clauses": ["1.1", "3.3", "4.2"],
  "routing_decision": "Manual review",
  "coverage_assessment": "Possibly covered",
  "confidence_level": "Medium",
  "latency_ms": 11234
}
```

11.5 Cost + latency budget (observed)

ส่วน	เป้าหมายความเร็ว	cost ต่อ triage
RAG retrieve	< 300 ms	$0 (local)
Claude Sonnet (triage)	5-10 s	~$0.015
Output guardrail + audit write	< 50 ms	$0
รวมต่อเคส	**~10-15 s**	**~$0.015**
Full 5-case eval + Judge (Opus)	~2 นาที	**~$0.80**

11.6 ตรวจสุขภาพระบบ (monitoring plan — ยังไม่ implement)

ตัววัด	เตือนเมื่อ
p95 latency	> 20 s
error rate	> 2%

citation validity	< 100%
guardrail trigger rate	พุ่ง 2 เท่าของค่าเฉลี่ย
coverage/routing accuracy (ถ้ามี manual labels)	ลด > 5 pp จาก baseline

Future: hook OpenTelemetry → Langfuse / Grafana (เป็น Phase 2+ ตาม `docs/limitations.md`)

11.7 เตรียมคำตอบเมื่อ regulator/compliance ถามเรื่อง accountability

> ***Compliance:*** *"AI มั่วแล้วเราผิดไหม?"*
>
> ***คำตอบที่พร้อมอธิบายได้:***
>
> 1. *AI มี guardrails 3 ชั้น — มีการตรวจ cite ทุก decision*
> 2. *ทุก triage มี audit JSON — ตรวจย้อนได้ว่า AI อ้าง clause ไหน*
> 3. ***คนตัดสินสุดท้ายทุกเคส** — AI แค่ recommend*
> 4. *Low confidence บังคับ Manual review — ไม่มีทางหลุดไป auto-approve*
> 5. *Prompt + model version บันทึก → reproduce output เดิมได้*
> 6. *100% citation validity บน 5 PDF test cases (verifiable)*

---

12. แผนทำจริง

12.1 MVP (3 วันของ assignment — ส่งแล้ว บน `develop`)

- ✅ Claim Triage Agent แบบ text (ตาม PDF §6)
- ✅ Policy RAG (Qdrant + BGE-small-en)
- ✅ Pydantic structured output + Anthropic forced tool-use
- ✅ Guardrails 3 ชั้น
- ✅ 5 test cases ตาม PDF §7
- ✅ LLM-as-Judge (Opus 4.7) 5-dim rubric
- ✅ Audit log per session
- ✅ Streamlit UI + CLI entry points
- ✅ README + Dockerfile + Makefile
- ✅ Solution design + implementation plan + limitations + prompts docs

12.2 Phase 2 (1-2 เดือน — อยู่บน branch `extensions` แล้วบางส่วน)

- ✅ Voice intake prototype (Streamlit mic + Whisper + Edge-TTS)
- ✅ SupervisorAgent multi-agent pattern + conversation state
- ✅ Multilingual RAG (e5-large) + 14 bilingual test cases
- ✅ Call transcript log + .txt export
- ⬜ Twilio telephony integration (ไม่ทำใน `extensions` เพราะ regulatory + cost)
- ⬜ Officer workflow — override + reroute controls บน UI
- ⬜ Prompt caching (Anthropic cache_control) บน policy context

12.3 Phase 3 (3-6 เดือน)

- OCR + vision Claude ประเมินรูปความเสียหาย
- Fine-tuned Thai STT (เช่น Typhoon-ASR)
- โมเดลจับฉ้อโกงเฉพาะทาง (บน data จริง)
- A/B test เทียบกับ manual triage baseline
- หัวหน้าตรวจ sample 5% workflow
- MCP SharePoint สำหรับประกาศ internal policy update

12.4 Phase 4 (6 เดือน+)

- เชื่อม core claim system จริง (one-way read ก่อน)
- AI เสนอวงเงิน settlement (ยัง HITL · ไม่ auto-approve)

- Predictive triage ตั้งแต่ sensor จับได้ (connected car)

---

13. ลำดับการพรีเซนต์

> ***สิ่งที่ rubric ต้องการ:*** *พูดชัดเจน · ตรง scope PDF · มีตัวเลข eval สนับสนุน*

13.1 ลำดับการพูด (10 นาที)

```
[0:00–1:30] เปิดด้วย pain story (officer perspective)
  → "พนักงานต้องเปิด policy อ่าน + ตรวจเอกสาร + สรุปทุกเคสเอง"
  → ชี้ที่ P1–P6 pain table
  → "AI นี้ไม่ได้แทนที่พนักงาน — ช่วยให้เร็วและสม่ำเสมอ"

[1:30–2:30] แมป pain → feature → metric
  → ตารางที่ §1.3 — ทุก pain มี feature + ตัววัด
  → ย้ำ: PDF §4 "In Scope" ครบ · "Out of Scope" ไม่แตะ

[2:30–4:00] ภาพรวมวิธีการทำงาน
  → Journey ของ officer (§3.2)
  → ย้ำ "AI = ผู้ช่วย · officer = decision maker"
  → แสดง UI screenshot (§3.3)

[4:00–6:30] สถาปัตยกรรม
  → ภาพ architecture (§5.1)
  → อธิบาย RAG (section-aware chunk, 4 sub-queries, metadata filter)
  → อธิบาย forced tool-use + Pydantic shape guarantee
  → ทำไมเลือก Sonnet (reasoning), Opus (judge), bge-small-en (ขนาดเหมาะ)

[6:30–8:00] Live demo
  → รัน `make demo CASE=case2_illegal_racing` หน้าจอ
  → แสดง output เต็ม + citations
  → เปิด audit JSON ให้ดู
  → แสดง 5-case eval report หน้า `docs/eval_report.md`

[8:00–9:00] Risk & Governance
  → Guardrails 3 ชั้น (§11.2)
  → HITL (§11.3)
  → Audit trail — "ตอบ regulator ได้" (§11.7)
  → ตัวเลข headline: **100% routing, 100% citation validity, 80% coverage, 4.00/5 judge**

[9:00–10:00] Roadmap + ปิด
  → Phase 1 (develop · shipped) → Phase 2 (extensions · มี POC แล้ว)
  → Phase 3–4 roadmap
  → ปิด: "AI ตัวนี้ลด cycle time ของ officer โดยไม่กระทบ HITL
     และตอบ compliance ได้ทุก decision — ตรง PDF scope ครบ"
```

13.2 คำถามที่ผู้ตรวจน่าจะถาม (เตรียมคำตอบไว้)

คำถาม	คำตอบ
ทำไมไม่ใช้ GPT-4o หรือ Gemini?	Claude Sonnet 4.6 เด่น tool-use adherence + reasoning + raw cost reasonable + context ยาว แต่ architecture decoupled เปลี่ยนโมเดลได้ด้วย env var
AI มั่วแล้วถูกฟ้อง?	1) ทุก coverage ต้อง cite 2) output guardrail post-check 3) คนคือผู้ตัดสิน 4) audit JSON ครบ 5) 100% citation validity ที่พิสูจน์ได้

10-15 วิต่อเคสยอมรับได้ไหม?	เทียบ manual review ที่ 10-30 นาทีต่อเคส → ดีขึ้น 100x · ถ้าต้องเร็วกว่านี้ switch ไป Haiku รุ่นเล็ก (trade-off accuracy)
cost scale?	~$0.015/triage × volume → ถ้าสูง → Haiku + prompt caching + batch
ถ้า Anthropic ล่ม?	fallback หลาย provider (OpenAI/Bedrock proxy) · code decoupled via SDK interface
Policy เปลี่ยนบ่อยทำยังไง?	Edit `data/policies/motor_policy.md` → `make ingest` → re-embed → ใช้ได้ทันที (no retrain needed)
ทำไม 5 cases พอ?	PDF ขอ 3+ (strong = 5) · 5-case eval + 5-dim judge = ครอบคลุม coverage/routing/missing/risk/reasoning · เพิ่ม cases ได้ง่าย

สรุป

Solution นี้ตอบ PDF **Motor Insurance Claim Triage Assistant** โดย:

- รับ ClaimInput ตาม PDF §5.2
- Ground ด้วย Policy Rules ตาม PDF §5.1 (RAG + 4 sub-queries)
- Produce TriageOutput ตาม PDF §6 (7 required fields + 1 extra permitted)
- Test ครบ 5 cases ตาม PDF §7 → **100% routing / 100% citation validity / 4.00/5 judge**
- Not covered + Guardrails 3 ชั้น + Audit JSON → อยู่ใน PDF §4 In Scope ครบ
- ไม่แตะ PDF §4 Out of Scope (ไม่ auto-approve, ไม่ชำระเงิน, ไม่ติด core system จริง, ไม่ customer-facing)

Voice / Multi-agent / Thai / multi-turn intake dialogue — **อยู่ใน branch `extensions`** ให้ดู เป็น Phase 2+ roadmap

---

> ***หมายเหตุ:*** *เอกสารนี้เป็น POC design ตัวเลข eval บน 5 PDF cases บน `develop` branch · ตัวเลข projection (AHT -30%, fraud recall +15%) เป็น estimate จาก industry benchmark · validated ที่ 100% routing + 100% citation validity ใน 5-case harness*